



Controlling Job Execution Order in GitLab CI/CD

Previous Job Conditions & Advanced Run Rules

Agenda

1 Why Control Job Order?

Understanding the importance of job sequencing in CI/CD pipelines

2 Using needs for Explicit Dependencies

Creating direct job dependencies regardless of stage

3 when: & allow_failure: Controls

Configuring job execution conditions and failure handling

4 "Run Anyway" & Failure-Tolerant Jobs

Setting up resilient jobs that execute regardless of upstream status

5 Advanced rules: Integration

Combining conditional execution with complex pipeline configurations

Why Control Job Order?



Enforce Correct Sequencing

Ensure build → test → deploy happens in the right order, preventing deployment of untested code

Balance Speed & Dependencies

Optimize pipeline runtime by running independent jobs in parallel while respecting critical path dependencies

Handle Support Tasks

Reliably execute cleanup, notifications, or audit steps at appropriate pipeline points

Graceful Failure Management

Prevent non-critical failures from breaking the entire pipeline while ensuring critical failures halt execution

Explicit Dependencies with needs

```
job1: stage: build  script: - echo "Build  
step"job2: stage: test  script: - echo "Test step"  
needs: [job1]
```

With **needs**, job2 will only run after job1 completes successfully, regardless of stage placement. This creates a direct dependency between specific jobs.



Standard Pipeline Flow

Without needs, jobs execute stage-by-stage (build → test → deploy)



Cross-Stage Dependencies

With needs, jobs can wait for specific jobs from any previous stage



Non-Linear Pipelines

Create complex workflows with parallel execution paths and custom dependencies

Controlling Run Conditions

1

when: on_success (default)

Job runs only if all jobs from earlier stages succeed

This is the default behavior if no **when:** is specified

2

when: on_failure

Job runs only when at least one job from earlier stages fails

Perfect for failure notifications or recovery tasks

3

when: always

Job runs regardless of the status of earlier jobs

Ideal for cleanup operations or status reporting

4

when: manual, delayed, never

Manual: requires user intervention to start

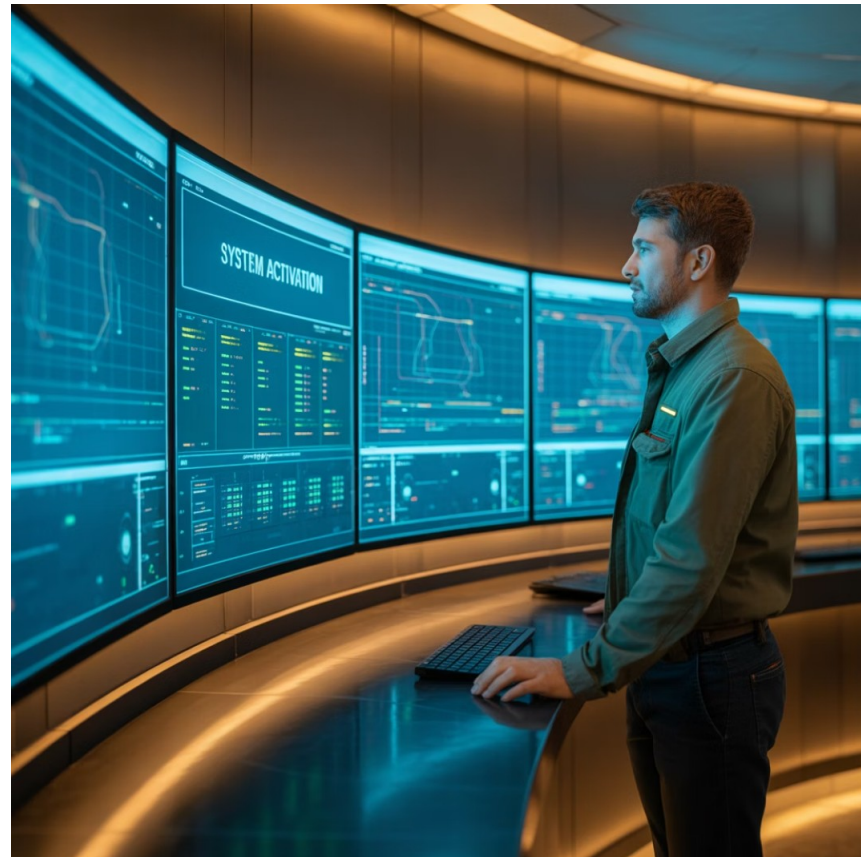
Delayed: runs after a specified time delay

Never: won't run (typically used with **rules:**)

 **allow_failure:** When set to **true**, lets a job fail without failing the entire pipeline. Default is **false**.

"Run Anyway" Jobs

```
build: script: exit 1 # simulates failure  
notify: script: echo "Notify stakeholders" when: always
```



Key Benefits:

- The notify job runs regardless of the build job's outcome
- Ensures critical operations always execute, even during pipeline failures
- Maintains system integrity with proper cleanup operations
- Guarantees stakeholder communication at all times

Common Use Cases:

- Sending notifications about pipeline status
- Performing environment cleanup after tests
- Generating reports regardless of test outcomes
- Logging final pipeline state for audit purposes

Failure-Tolerant Jobs

```
test_job: script: exit 1 allow_failure: true deploy: script: echo  
"Deploying..."
```

In this example, the pipeline continues to the deploy job even though test_job fails, because we've set allow_failure: true on the test_job.



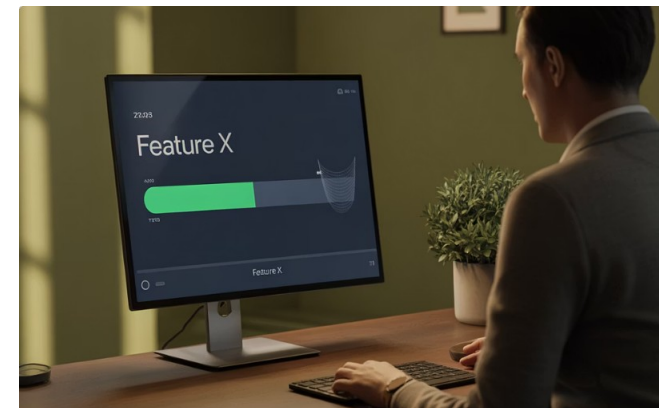
Non-Critical Tests

Allow experimental or supplementary tests to fail without blocking deployment



Quality Gates

Run code quality checks that warn but don't prevent progression



Staged Rollouts

Deploy to certain environments even if others fail, allowing partial releases

Quick Reference

Goal	Configuration	Example Use Case
Run only on success	Default behavior (when: on_success)	Standard deployment that requires all tests to pass
Run regardless of outcome	when: always	Cleanup scripts, notifications, logging
Allow job to fail without breaking pipeline	allow_failure: true	Experimental tests, non-critical quality checks
Run only if any previous failed	when: on_failure	Failure notifications, recovery operations
Require manual intervention	when: manual	Production deployments requiring approval
Run after specific jobs only	needs: [job1, job2]	Multi-path pipelines with specific dependencies

Combine these options to create sophisticated pipeline control flows tailored to your project's needs

Advanced: Combining with rules:

```
audit:  script: echo "Audit logs"  rules:    - if:
'$CI_PIPELINE_SOURCE == "schedule"'    when: always    - when:
never
```

Note: When both `rules:` and `when:` are present, the `when:` inside `rules:` takes precedence for matched conditions.

Complex Conditions

Use `rules:` to evaluate branch filters, variables, and schedule triggers before determining job execution

Conditional When

Apply different `when:` values based on specific conditions, creating context-aware execution

Precedence Rules

The `when:` inside a matched `rule:` condition overrides any top-level `when:` directive

Environment-Specific Behavior

Create jobs that behave differently depending on branch, schedule, or variable conditions



Thank you

